

JavaScript Foundations Quiz

Test your fundamental knowledge of JavaScript data types, scope, async operations, and ES6+ features.

Name: _____

Date: _____

Time Allowed: 20 Minutes

Total Questions: 10

Q1. What is the output of the following JavaScript expression?

```
console.log(typeof null);
```

- ☐ A) "null" ☐ B) "undefined"
- ☐ C) "object" ☐ D) "boolean"

Q2. Which keyword creates a block-scoped variable that cannot be reassigned?

- ☐ A) `var` ☐ B) `let`
- ☐ C) `const` ☐ D) `static`

Q3. What will be printed to the console after executing this code?

```
console.log(1 + "2" - 1);
```

- ☐ A) 12 ☐ B) 11
- ☐ C) 2 ☐ D) NaN

Q4. What is the primary difference between the `==` and `===` operators?

- ☐ A) `==` checks type, `===` checks value ☐ B) `===` performs type coercion, `==` does not
- ☐ C) `===` checks both value and type without coercion ☐ D) There is no difference in JavaScript

Q5. What will be the output of the following arrow function execution?

```
const obj = {  
  num: 10,  
  getNum: () => this.num  
};  
console.log(obj.getNum());
```

- ☐ A) 10 ☐ B) undefined
- ☐ C) ReferenceError ☐ D) null

Q6. Which array method creates a new array populated with the results of calling a provided function on every element?

- ☐ A) `forEach()` ☐ B) `filter()`
☐ C) `map()` ☐ D) `reduce()`

Q7. What is the output of this Promise chain?

```
Promise.resolve(1)
  .then(x => x + 1)
  .then(x => { throw new Error('Err'); })
  .catch(() => 5)
  .then(x => console.log(x));
```

- ☐ A) Error: Err ☐ B) 2
☐ C) 5 ☐ D) undefined

Q8. Which method converts a JSON string into a corresponding JavaScript object?

- ☐ A) `JSON.stringify()` ☐ B) `JSON.parse()`
☐ C) `JSON.toObject()` ☐ D) `Object.fromJSON()`

Q9. What term describes a function bundled together with references to its surrounding state (lexical environment)?

- ☐ A) Prototype ☐ B) Generator
☐ C) Closure ☐ D) Callback

Q10. What is the default value of a variable declared with `let` that has not been initialized?

- ☐ A) `null` ☐ B) `undefined`
☐ C) `0` ☐ D) `false`

Answer Key & Explanations

Q1: C ("object")

In JavaScript, `typeof null` returns `"object"`. This is a historical bug from the initial implementation of JavaScript, where null was represented with a null pointer tag matching objects.

Q2: C (const)

`const` creates block-scoped variables that cannot be reassigned after declaration. `let` creates block-scoped variables that can be reassigned, whereas `var` is function-scoped.

Q3: B (11)

Evaluated left-to-right: `1 + "2"` performs string concatenation resulting in `"12"`. Next, `"12" - 1` uses numeric subtraction, coercing `"12"` to the number 12, giving `11`.

Q4: C (=== checks both value and type without coercion)

The loose equality operator `==` performs type implicit coercion if the types differ, whereas the strict equality operator `===` checks both value and type without coercion.

Q5: B (undefined)

Arrow functions do not bind their own `this`; instead, they inherit `this` lexically from the enclosing scope. In this case, `this` refers to the outer global/window context where `num` is undefined.

Q6: C (map())

`Array.prototype.map()` returns a brand new array containing transformed items evaluated by the callback function. `forEach()` executes a callback for side effects without returning a new array.

Q7: C (5)

The thrown error triggers the `.catch()` block, which returns 5. A `.catch()` block returns a newly resolved Promise holding the returned value, passing 5 to the subsequent `.then()`.

Q8: B (JSON.parse())

`JSON.parse()` parses a JSON-formatted string into a JavaScript object. `JSON.stringify()` performs the reverse transformation.

Q9: C (Closure)

A closure is a feature in JavaScript where an inner function retains access to variables from its outer (enclosing) lexical scope even after the outer function has finished executing.

Q10: B (undefined)

Variables declared using `let` or `var` without an explicit initial value are automatically assigned the default value of `undefined` by JavaScript runtime engine.